

COMP601 - ASSIGNMENT

LEARNING OUTCOMES

The course work tasks aim to practice the following topics:

- Basic input and output operations
- Coding standards
- Debugging and testing by using features of Integrated Development Environment (IDE)
- The use of methods (both static and instance) and parameters in solving problems
- Design and implement classes and objects.
 - public and private class fields
 - constructors, public, and private methods
 - Modular programming
 - Problem solving using classes and objects.
- Derived classes and protected variables and methods: Examples and problem solving.
- Polymorphism and its advantages in software development: Examples and problem solving.
- Using object oriented programming to develop applications.
- Object oriented programming and code reuse
- Array of objects
- Object persistence with file I/O

MARKING PROCESS

STEP 1 Copy all your **.java** files into the provided upload textbox on Moodle.

STEP 2 Demonstrate your solution to your tutor during class. You may have to explain to your tutor how your program works and be prepared to answer questions with regards to your solution.

MARKING CRITERIA AND MARK ALLOCATION

To achieve the allocated mark on each question, your solution must be complete and function correctly according to the following:

- The question requirement description.
- The sample run outputs (if any).
- Your program must follow the **Programming Best Practices** – you can find the document on Moodle.

TASKS TO COMPLETE

Note, there is a marking guide at the end of this document.

TASK 1 – FORMATTED OUTPUT

- Learning outcomes: 1, 3
- Relevant topics: output, escape sequence characters, composite formatting
- Resources: self-learning tutorials videos, lecture tutorials, reference material on Moodle, recommended textbooks
- Moodle section reference: Data types and operators

Create a program that displays the output given below. You will need to make use of character quoting to formulate and display the Escape Sequence characters correctly.

Hints: The backslash (\) and the double quotes (") are both special characters. To make them appear in a string as normal characters, both need a backslash in front of them. For example, "\\\" gives a single backslash as a normal character, whereas "\" gives a double quotes character.

Note: You are required to use formatted output to space out the columns. To line up the columns, do not manually insert whitespace characters, do not use the tab combination \t. Use printf and formatting template to achieve column width instead.

The following is a sample run output:

Escape sequence	Description

\n	New line character
\t	Horizontal tab character
\"	Double quote character
\b	Backspace character

TASK 2 – ARITHMETIC CALCULATION

- Learning outcomes: 1, 3
- Relevant topics: input, output, arithmetic operators, composite formatting
- Resources: self-learning tutorials videos, lecture tutorials, reference material on Moodle, recommended textbooks
- Moodle section reference: Data types and operators

Write a program that asks from the user for the radius of a circle. Calculate circle area and perimeter and display the results using printf and formatting template (round up results to 2 decimal points).

Formula for calculating circle area and perimeter are as follows:

$$\text{Area} = \text{radius}^2 * \pi$$

$$\text{Perimeter} = \text{diameter} * \pi$$

$$\text{where, } \text{diameter} = \text{radius} * 2, \pi = \text{Math.PI}$$

Sample run output are as follows:

Enter circle radius: 2.5

Radius	Perimeter	Area

2.5	15.71	19.63

TASK 3 – USER INPUT VALIDATION

- Learning outcomes: 1, 3
- Relevant topics: input, output, iteration
- Resources: self-learning tutorials videos, lecture tutorials, reference material on Moodle, recommended textbooks
- Moodle section reference: Selection and iteration

Write a program that asks the user for a score (as double). If the user input number falls outside 0 – 100, the program must prompt for re-entry until a valid score is received. The program should display a letter grade based on the user entered score, as follows:

Score	Grade
0-50 (including 0 but not 50)	D
50-70 (including 50 but not 70)	C
70-80 (including 70 but not 80)	B
80-100 (including both 80 and 100)	A

Sample run as follows:

Enter a score: 120

Invalid score

Enter a score: -10

Invalid score

Enter a score: 75.5

Score 75.5 will receive grade B

TASK 4 – ARRAY AND LIST

- Learning outcomes: 1, 3
- Relevant topics: input, output, iteration, array
- Resources: self-learning tutorials videos, lecture tutorials, reference material on Moodle, recommended textbooks
- Moodle section reference: Array and List

Write a program to ask the user to enter two lines of colour names, separated by comma. The program then calculates the number of overlapping colour names between the two lines.

The following is a suggested approach:

1. Read two strings, with each containing a bunch of colour names.
2. Split each string into an array, with each array containing the corresponding colour names.
3. Create an empty list of strings, which will be used to store the duplicate colour names in later step.
4. Use two nested for loops to iterate the two arrays and add the duplicate colour names into the list created in the previous step.
5. The `list_name.size()` method can be used to retrieve the number of common colour names in the list.
6. Use a simple for loop to display the content of the list to output the common colour names.

Note:

- You do not have to perform user input checking.
- You can assume the colour names within each line are unique.

Sample run output as follows:

Enter first line of colours: **red,blue,green,purple,black**

Enter second line of colours: **green,red,grey,white**

You entered 5 colours in first list

You entered 4 colours in second list

Number of overlapping colours: 2

Overlapping colours are as follows:

red

green

TASK 5 - CLASS AND INSTANCE

- Learning outcomes: 1, 3
- Relevant topics: input, output, iteration, array, list, class, and instances
- Resources: self-learning tutorials videos, lecture tutorials, reference material on Moodle, recommended textbooks
- Moodle section reference: Class and instance

Create a class `Note`, as in note in music. A `Note` object is represented by a pitch (one of the seven letters – C, D, E, F, G, A, and B – which is the simplest and most common way of specifying a music note), and a duration (in number of seconds). Write the class `Note` that contains

- Private class fields with getter/setter: pitch (String), duration (int).
- A parameterized constructor that takes two parameters and sets the fields.
- A public method `playNote()` that outputs the pitch and the duration of a note in the format as in the sample output below (i.e., "The note <pitch letter> is played for <duration> seconds"). Method header is given as following: `public void playNote()`
- The code in `main()` is given below. A list of `Note` instances were created and passed as parameter to the methods. Complete the method `processNotes`.

```
public class Main {
    void main() {
        List<Note> notes = new LinkedList<>();
        notes.add(new Note("D", 19));
        notes.add(new Note("C", 20));
        notes.add(new Note("F", 31));
        notes.add(new Note("B", 45));
        notes.add(new Note("C", 73));
        notes.add(new Note("F", 13));
        notes.add(new Note("B", 34));
        notes.add(new Note("C", 53));

        processNotes(notes);

    } //end of main method

    public static void processNotes(List<Note> notes){
        // Complete method
    }
} //end of class Main
```

The **processNotes** method displays each `Note` instance and calculates the total duration value. See sample run output below. Hint, use a loop to iterate the "notes" list and invoke `playNote()` method on each instance; use a total variable to accumulate the duration of each instance (this can be done in the same loop).

Sample run output:

```
The note D is played for 19 seconds
The note C is played for 20 seconds
The note F is played for 31 seconds
The note B is played for 45 seconds
The note C is played for 73 seconds
The note F is played for 13 seconds
The note B is played for 34 seconds
The note C is played for 53 seconds
```

Total duration: 288 seconds

TASK 6 – INHERITANCE

- Learning outcomes: 1, 3
- Relevant topics: input, output, iteration, array, list, superclass, subclass
- Resources: self-learning tutorials videos, lecture tutorials, reference material on Moodle, recommended textbooks
- Moodle section reference: Inheritance

Create class Vegetable, with:

- Private fields with getter/setter: weight (double, measured in kg), price (double, as in \$NZ dollars per kg)
- A parameterized constructor that takes two parameters.

Create class LeafyGreen, as subclass of Vegetable. It contains:

- Private field with getter/setter: type (string, as in one of three leafy green vegetable types: cabbage, Lettuce, broccoli)
- A parameterized constructor that takes three parameters. Call superclass constructor using the “super” keyword.

The code in `main()` is given below. A list of LeafyGreen instances are created and passed as parameter to the method `calculateStats()`.

```
public class Main {
    void main() {
        List<LeafyGreen> veges = new LinkedList<>();
        veges.add(new LeafyGreen(10, 3.5, "Cabbage"));
        veges.add(new LeafyGreen(5, 2.94, "Lettuce"));
        veges.add(new LeafyGreen(20, 5.45, "Broccoli"));
        veges.add(new LeafyGreen(7, 1.94, "Lettuce"));
        veges.add(new LeafyGreen(15, 2.8, "Lettuce"));
        veges.add(new LeafyGreen(30, 2.5, "Cabbage"));
        veges.add(new LeafyGreen(28, 4.4, "Broccoli"));
        veges.add(new LeafyGreen(20, 5.5, "Cabbage"));
        veges.add(new LeafyGreen(16, 3.9, "Broccoli"));
        veges.add(new LeafyGreen(26, 4.8, "Broccoli"));

        calculateStats(veges);
    } //end of main

    public static void calculateStats(List<LeafyGreen> veges) {
        // Complete this method to produce the sample output below
    }
} //end of class Main
```

Note, the first instance in the list, `new LeafyGreen(10, 3.5, "Cabbage")`, its weight is 10 kg, price is \$3.5/kg, vege type is Cabbage. Its cost is calculated as: 10kg x \$3.5/kg = \$35.0. The method `calculateStats()` calculates the following:

- the total cost of all instances in the list
- the total weight of each of the three types: Cabbage, Broccoli, and Lettuce

Sample run output:

```
Total vegetable cost: $709.7
Total Cabbage weight: 60.0 kg
Total Broccoli weight: 90.0 kg
Total Lettuce weight: 27.0 kg
```

TASK 7 – POLYMORPHISM

- Learning outcomes: 1, 3
- Relevant topics: input, output, iteration, array, list, superclass, subclass
- Resources: self-learning tutorials videos, lecture tutorials, reference material on Moodle, recommended textbooks
- Moodle section reference: Polymorphism

Create a class Shape, that contains:

- Private field with getter/setter: colour (String).
- A parameterized constructor that takes a single parameter and sets the field.
- A method getShapeType() that returns the message "I'm a Shape".

Method header: `public String getShapeType()`

Create a class Rectangle, as subclass of Shape. It contains:

- Private fields with getter/setter: height (double), and width (double).
- A parameterized constructor that takes three parameters. Call superclass constructor using the "super" keyword.
- An override method getShapeType() that returns the message "I'm a Rectangle".

Method header: `@Override public String getShapeType()`

- A method getPerimeter() that calculates and returns the rectangle perimeter.

Method header: `public double getPerimeter()`

In the main() method below, two arrays of objects are created. Write the rest of code to produce the sample run output:

```
public class Main {
    void main() {
        List<Shape> shapes = new LinkedList<>();
        shapes.add(new Shape("white"));
        shapes.add(new Rectangle("red", 10, 6));
        shapes.add(new Rectangle("black", 20, 9));
        shapes.add(new Shape("orange"));
        showShapeNames(shapes); //produces output part 1

        Rectangle[] rectangleArray1 = {
            new Rectangle("white", 4, 3),
            new Rectangle("red", 9, 5),
            new Rectangle("purple", 3, 6),
            new Rectangle("orange", 15, 10),
            new Rectangle("black", 4, 14),
        };
        Rectangle[] rectangleArray2 = {
            new Rectangle("pink", 3, 4),
            new Rectangle("red", 10, 2),
            new Rectangle("white", 8, 5),
            new Rectangle("blue", 14, 4),
            new Rectangle("bindle", 10, 15),
        };
    }
}
```

```

        //produces output part 2
        countOverlapRectangles(rectangleArray1, rectangleArray2);
    }//end of main() method
    public static void showShapeNames(List<Shape> shapes){
        //Complete method to produce sample run output part 1
        //Hint: iterate list "shapes", invoke getShapeType() on each instance

    }
    public static void countOverlapRectangles(Rectangle[] group1, Rectangle[]
group2){
        // Complete method to produce sample run output part 2
        // Note: you can assume that the colours/perimeters in each group are unique,
        // i.e., there are no duplicate colours/perimeters within each group.
    }
}
} //end of Main class

```

Sample run output part 1:

```

I'm a Shape
I'm a Rectangle
I'm a Rectangle
I'm a Shape

```

Sample run output part 2:

```

There are 2 Rectangle objects with overlapping colour between the two arrays
There are 3 Rectangle objects with overlapping perimeter between the two arrays

```

TASK 8 – RECURSIVE PROGRAMMING

- Learning outcomes: 1, 3
- Relevant topics: input, output, iteration, array, list, class, instance, recursion
- Resources: self-learning tutorials videos, lecture tutorials, reference material on Moodle, recommended textbooks
- Moodle section reference: Recursion

Create a class Bag that contains

- Private fields with getter/setter: colour (String), capacity (double)
- A parameterized constructor that takes two parameters and sets the fields

Create a class BagApp. Use the template given below. Complete the recursive method.

The code for class Main are given below, which creates a list of Bag objects; the list is then passed to the method calcTotalCapacity.

```
public class Main    {
    void main()    {
        List<Bag> bags = new LinkedList<>();

        //fill in code here: create a list of Bag instances
        //and add to the list "bags".

        BagApp bgapp = new BagApp();
        double totalCapacity = bgapp.calcTotalCapacity(bags);
        println(String.format("Total capacity: %.2f", totalCapacity));

        println("Total number of bags: " + bags.size());
    }
} //End of class Main

public class BagApp {
    public double calcTotalCapacity(List<Bag> bagList) {
        //Write the code for this recursive method
        //This calculates and returns the total capacity of all instances in "bagList".
        //Hint: make use of the capacity getter: getCapacity().
    }
} //End of class BagApp
```

A sample run is shown below:

Total capacity: 12.90

Total number of bags: 3

TASK 9 – GUESSING GAME

- Learning outcomes: 1, 3
- Relevant topics: input, output, iteration, array, list, class, instance, file input and output
- Resources: self-learning tutorials videos, lecture tutorials, reference material on Moodle, recommended textbooks
- Moodle section reference: File I/O and random number generation

Create a class Country, with:

- Private class fields with getter/setter: name (string), capital (string), population (int)
- A parameterized constructor with three parameters.

Use the text below to create a data file data.txt. In the data file, each line contains the country name, the capital city name, and the country population (in million).

```
New Zealand,Wellington,5
India,New delhi,1380
Germany,Berlin,83
Thailand,Bangkok,69
Greece,Athens,10
Netherlands,Amsterdam,17
```

Create a class GameApp (code template given further below):

- Write method **readData()** to read the data file content and populate the list of Country instances.
- Write method **game()** that asks the user to guess the capital city of a country randomly selected from the list. For example, "What's the capital city of New Zealand?" "Wellington".
- **Optional task 1:** enhance the game so that it keeps running until the user chooses to stop. For example, "Play again (y/n)?" "y".
- **Optional task 2:** enhance the game to allow the user three attempts before revealing the correct answer.

In the readData() method, create and store the Country instances in a list. In the game() method, first generate a random number in the range of 0 up to the list size; then use this number as index to retrieve an instance from the list. The name of the Country instance is used to ask the question, and the capital of the instance is used to check the answer by the user.

Check for previous lectures for how to generate random numbers in a range.

The following is a suggested template:

```
public class Main {

    void main() {
        String txtfile = "src/<your-package-name>/data.txt";
        try {
            GameApp app = new GameApp(txtfile);
            app.game();

        } catch (IOException ioe) {
            println(String.format("Perhaps missing text file: %s/%s?",
                                new Main().getClass().getPackage().getName(), txtfile));
        }
    } //End of main method
} //End of class Main

public class GameApp {
    private List<Country> countryList = new LinkedList<>();
    public GameApp(String filename) throws IOException {
        readData(filename);
    }
    public void readData(String filename) throws IOException {
        // write the code
    }
    public void game() {
        // write the code
    }
} //End of class GameApp

public class Country{
    // write the class code
} //End of class Country
```

Sample run 1:

What's the capital of New Zealand? Hamilton

Incorrect. It's Wellington.

New Zealand is a country of 5 million.

Sample run 2:

What's the capital of India? new dehli

Well done!

India is a country of 1380 million.

TASK 10 – PROCESSING BOOK LIST

- Learning outcomes: 1, 3
- Relevant topics: input, output, iteration, array, list, class, instance, file input and output
- Resources: self-learning tutorials videos, lecture tutorials, reference material on Moodle, recommended textbooks
- Moodle section reference: File I/O

Create a class `Book`, with:

- Private class fields with getter/setter: `name` (string), `year` (int, as in publishing year)
- A parameterized constructor with two parameters.

Create a class `AudioBook`, as subclass of `Book`, with:

- Private class fields with getter/setter: `length` (double, as in audio book length in hours).
- A parameterized constructor with three parameters.
- A void method `displayInfo` that displays the book name, year, and length.

Now manually create a data file. Use the text below as the data file content. The text contains some `AudioBook` records, each record contains the book name, publishing year, and the audio book length (in hours). These are separated by comma, with one record per line.

```
Killing floor,1997,15.4
Zero day code,2019,12.5
The killer across the table,2019,9.6
West cork,2018,11
Sleep walk,1985,10.3
The end of the road,2001,9.6
I'll be gone in the dark,2018,9.9
```

Create a class `AudioBookApp` (code template given further below):

1. Write method **`readData()`** to read the data file content and populate the list of `AudioBook` instances. Hint: read the file content, line by line, split each line into an array of items, use the items to an `AudioBook` instance, add the instance to the list.
2. Write method **`getShortestBooks()`** that calculates and returns a list of `AudioBook` instances with the shortest length. Note: this is a value-return method, i.e., it must return a list of `AudioBook` instances back to the main.
3. Write method **`countBooks()`** that counts the number of `AudioBook` instances that were published during the period specified in the main. Note: this is a value-return method, i.e., the method should return the count at the end.
4. Write method **`randomBookList()`** that divides the `Book` instance list into three random lists. First create three empty lists (e.g., `b1`, `b2`, `b3`). Use a loop to iterate the pool list (i.e., `bookList`), for each instance, to decide which of the three lists it goes into, generate a random number between 1-3 and use the number to decide which list to add the instance in. Note that, you'll have to create as many random numbers as the number of available books, i.e., the code to generate a random number needs to be inside the loop that iterates the existing book instances.

Hint, there is function named `luckyDraw` in Lecture week 5 which is a useful reference for this task.

In class `Main`:

1. Write method **`displayBooks()`** that displays a list of book instances.

The following is a suggested template:

```
public class Main {

    void main() {
        String txtfile = "src/<your-package-name>/data.txt";
        try {
            AudioBookApp app = new AudioBookApp(txtfile);
            println("Shortest books:");
            displayBooks(app.getShortestBooks());
            println("Books published btw 2010-2020: " + app.countBooks());
            app.randomBookList();
        } catch (IOException ioe) {
            println(String.format("Perhaps missing text file: %s/%s? \n\n",
                                   new Main().getClass().getPackage().getName(), txtfile));
        }
    } //End of main method
    public void displayBooks(List<AudioBook> list) {
        // Write code
    }
} //End of class Main

public class AudioBookApp {
    private List<AudioBook> bookList = new LinkedList<>();
    public List<AudioBook> getBookList() {
        return bookList;
    }
    public AudioBookApp(String filename) throws IOException {
        readData(filename);
    }
    public void readData(String filename) throws IOException {
        // write the code
    }
    public List<AudioBook> getShortestBooks() {
        // write the code
    }
    public int countBooks() {
        // write the code
    }
    public void randomBookList() {
        // write the code
    }
} //End of class App

public class AudioBook extends Book {
    // write the class code
} //End of class AudioBook

public class Book {
    // write the class code
} //End of class Book
```

Sample run output:

Shortest books:

Title	Year	Length

The killer across the table	2019	9.6
The end of the road	2001	9.6

Books published btw 2010-2020: 4

2 books in list #1:

Killing floor 1997 15.4

The end of the road 2001 9.6

3 books in list #2:

The killer across the table 2019 9.6

Zero day code 2019 12.5

Sleep walk 1985 10.3

2 books in list #3:

West cork 2018 11.0

I'll be gone in the dark 2018 9.9

Name:



Work submitted on Moodle

Score:

COMP601 ASSIGNMENT MARKING GUIDE

TASK 1 (3 marks)	
Program produces the required output	2
Correct use of printf method and formatting templates	0.5
Programming Best Practices	0.5

TASK 2 (3 marks)	
Read user input as circle radius	1
Correctly calculate circle perimeter and area	1
Program produces the output by using printf method and formatting templates	0.5
Programming Best Practices	0.5

TASK 3 (4 marks)	
Read a score from user (as appropriate data type) and validate user input	1.5
Program produces the output according to task criteria	2
Programming Best Practices	0.5

TASK 4 (10 marks)	
Read two lines of colour names entered by user	1
Split each line into an array of strings	1
Calculate and display the number of colour names in each line	2
Calculate and display the number of overlappings	2
Calculate and display the overlapping colour names	3.5
Programming Best Practices	0.5

TASK 5 (10 marks)	
Class Note: fields (with getter/setter), constructor, method displayNote	2
Method processNotes	7.5
Programming Best Practices	0.5

TASK 6 (10 marks)	
Class Vegetable: fields (getter/setter), constructor	2
Class LeafyGreen: fields (getter/setter), constructor	2.5
Method calculateStats()	5
Programming Best Practices	0.5

TASK 7 (10 marks)	
Class Shape: fields (with getter/setter), constructor. method getShapeName	1.5
Class Rectangle: fields (with getter/setter), constructor	2.5
Method getShapeName	
Method getPerimeter	
Method showShapeNames	1.5
Method countOverlapRectangles	4
Programming Best Practices	0.5

TASK 8 (10 marks)	
Bag class: fields with getter/setter, constructor	2
Correctly implement recursive and base case	7.5
Programming Best Practices	0.5

TASK 9 (15 marks)	
Class Country, with fields and getter/setter, constructor	2
Method readData()	5
Method game()	7.5
Programming Best Practices	0.5

TASK 10 (25 marks)	
Class Book, with fields and getter/setter, constructor	2
Class AudioBook, with fields and getter/setter, constructor, and displayInfo method	2.5
Method readData()	5
Method getShortestBooks()	5
Method countBooks()	5
Method randomBookList()	5
Programming Best Practices	0.5